

# Deploy da Nacionalização - Guia de Execução

Branch: feat/nacionalizacao -> main

Banco de dev: xxdletrjyjajtrmhwzev · Banco de produção: mrejpgcxaevooofyenzq

Documento gerado em: 08/08/2026

*Tudo que está descrito aqui já foi aplicado e testado no banco de dev.*

*Este guia cobre a subida para produção.*

## Antes de começar

- ☐ Backup do banco de produção. No painel do Supabase:  
Database -> Backups -> Create backup. Não pule esta etapa: várias migrações alteram RLS e uma delas troca uma constraint.
- ☐ Escolha uma janela de baixo movimento. Durante a troca de policies há uma janela de segundos em que gravações podem falhar.
- ☐ Confirme que você tem a senha do banco de produção em mãos (o CLI pede ao conectar).

## Passo 1 - Publicar o código

```
git checkout feat/nacionalizacao
git push origin feat/nacionalizacao
```

Abra o Pull Request de feat/nacionalizacao para main e faça o merge.

*O Vercel publica automaticamente ao detectar o merge em main.*

*Importante: publique o código depois das migrações (Passo 2) ou*

*imediatamente antes - o front novo espera a tabela permission\_rules*

*existir. Se o site subir antes do banco, o painel de permissões dá erro*

*ao carregar (o restante do app continua funcionando).*

Ordem recomendada: Passo 2 (banco) -> Passo 3 (functions) -> Passo 1 (código).

## Passo 2 - Migrações do banco

### 2.1 Verificar o que já está aplicado

```
npx supabase migration list --project-ref mrejpgcxaevooofyenzq
```

Compare com a lista abaixo e aplique só o que faltar.

## 2.2 Migrações da nacionalização

| #   | Arquivo                                 | O que faz                            |
|-----|-----------------------------------------|--------------------------------------|
| 141 | public_works_city_id                    | Adiciona city_id em obras + backfill |
| 142 | public_works_ambassador_rls             | RLS de gestor para obras             |
| 143 | work_media_ambassador_select            | Leitura de mídia de obras            |
| 144 | bairros_city_id_floresta                | city_id em bairros                   |
| 145 | bairros_insert_gestor                   | Gestor pode criar bairro             |
| 146 | measurements_ambassador_rls             | RLS de medições                      |
| 147 | payments_ambassador_rls                 | RLS de pagamentos                    |
| 148 | rental_properties_schema                | Tabelas de imóveis alugados          |
| 149 | rental_properties_rls                   | RLS de imóveis                       |
| 150 | rental_property_media_bucket            | Bucket de mídia                      |
| 151 | pavement_streets_city_id                | city_id em pavimentação              |
| 152 | pavement_streets_ambassador_rls         | RLS de pavimentação                  |
| 153 | pavement_streets_update_with_check      | Corrige WITH CHECK                   |
| 154 | services_city_id                        | city_id em serviços                  |
| 155 | services_ambassador_rls                 | RLS de serviços                      |
| 156 | rental_properties_location_geometry     | geography -> geometry                |
| 157 | rental_properties_street_number         | Campo número do imóvel               |
| 158 | directory_city_id_not_null              | city_id obrigatório                  |
| 159 | fix_get_invite_preview_ambiguous_column | Corrige RPC de convite               |
| 160 | get_my_pending_invite                   | Aviso de convite pendente            |
| 161 | rental_contracts_expected_end_date      | Previsão de encerramento             |
| 162 | rental_properties_title                 | Campo Título (com backfill)          |
| 163 | bairros_unique_per_city                 | Único por (nome, cidade)             |
| 164 | permission_rules                        | Tabela + função can_write()          |
| 165 | permission_rules_policies               | can_write() em 24 policies           |

## 2.3 Aplicar

```
npx supabase link --project-ref mrejgpcxaevoofyenzq
npx supabase db push
```

## 2.4 Atenção especial

Migração 158 (directory\_city\_id\_not\_null) falha se houver registros com city\_id nulo. Verifique antes:

```
select count(*) from public.directory where city_id is null;
```

Se retornar mais que zero, corrija esses registros antes de aplicar.

Migração 162 (rental\_properties\_title) faz backfill do título usando a secretaria ou o endereço, e só então marca a coluna como obrigatória. Se houver imóvel sem nenhum dos dois, ela falha.

Migração 163 (bairros\_unique\_per\_city) cria índice único por (lower(nome), cidade). Falha se já existirem bairros duplicados na mesma cidade:

```
select lower(name), city_id, count(*)
  from public.bairros
 group by lower(name), city_id
 having count(*) > 1;
```

## 2.5 Conferir depois de aplicar

```
-- deve retornar 24
select count(*) from pg_policies
 where schemaname = 'public'
    and (coalesce(qual, '') like '%can_write%'
        or coalesce(with_check, '') like '%can_write%');

-- deve retornar 0 (ninguém bloqueado no primeiro dia)
select count(*) from public.permission_rules;
```

## Passo 3 - Edge Functions

Funções criadas ou alteradas nesta leva:

send-ambassador-invite-email, accept-ambassador-invite, reverse-geocode e backfill-public-works-city.

```
yarn deploy:functions:nacionalizacao
```

*Não use o deploy:functions:prod para isso. Aquele script publica*

*outro conjunto (push, pagamentos e compartilhamento) e não tem nenhuma*

*função em comum com as desta leva - rodá-lo sozinho não publicaria nada*

*do que a nacionalização precisa.*

## Secrets necessários

O e-mail de convite de embaixador usa Resend. Confira se existem em produção:

```
npx supabase secrets list --project-ref mrejpgcxaevooofyenzq
```

| Secret | Para quê |
|--------|----------|
|--------|----------|

|                   |                                          |
|-------------------|------------------------------------------|
| RESEND_API_KEY    | Envio de e-mails                         |
| RESEND_FROM_EMAIL | Remetente (domínio verificado no Resend) |
| APP_URL           | Monta o link do convite                  |

*Pendência conhecida: no ambiente de dev o RESEND\_FROM\_EMAIL não está*

*configurado, e por isso o envio cai no remetente padrão e o Resend recusa com*

*403. Se o mesmo acontecer em produção, configure:*

>

```
`bash
```

```
npx supabase secrets set RESEND_FROM_EMAIL="Trombone Cidadão <contato@seudominio.com.br>"
--project-ref mrejpgcxaevooofyenzq
```

```
`
```

>

*O domínio precisa estar verificado no painel do Resend. Enquanto isso não*

*for feito, o convite continua funcionando - o master copia o link ou*

*compartilha por WhatsApp pela tela de convites.*

## Passo 4 - App Android (opcional)

Só se for publicar uma nova versão do app.

```
# APK/AAB apontando para PRODUÇÃO
yarn build:prod          # APK release
yarn build:prod:aab      # AAB para a Play Store (incrementa o versionCode)
```

Para gerar uma build de teste contra o banco de dev:

```
yarn build:standalone:dev    # APK
yarn build:standalone:dev:aab # AAB
```

*Os arquivos .env têm nomes enganosos: .env aponta para produção e*

*.env.production aponta para dev - e o Vite dá precedência ao .env. Por*

*isso os scripts :dev forçam --mode development. Não renomeie esses*

*arquivos sem revisar todos os scripts de build.*

## Passo 5 - Verificação pós-deploy

## Fluxos gerais

- ☐ Trocar de cidade no seletor e conferir que obras, imóveis, pavimentação e serviços filtram junto.
- ☐ Criar uma bronca: confirmar o aviso de que passará por moderação e aparecerá no feed depois de aprovada.
- ☐ Abrir uma bronca com vídeo no feed: deve tocar sozinho, mudo, com o botão de som no canto.
- ☐ Compartilhar uma bronca pelo app: o link não pode conter localhost.

## Embaixadores

- ☐ Gerar um convite e conferir que o link abre a tela de aceite.
- ☐ Após aceitar, o aviso de convite pendente deve sumir sem precisar recarregar a página.
- ☐ O embaixador só enxerga dados da cidade dele.

## Painel de permissões (novo)

- ☐ Entrar como master e abrir /admin/permissoes.
- ☐ Desligar Obras no cargo Embaixador.
- ☐ Com um embaixador: o botão "Adicionar obra" some, /obras/gerenciar redireciona e o item some do menu "Gerenciar". Os outros módulos continuam normais.
- ☐ Criar exceção liberando Obras só para esse embaixador: o acesso volta.
- ☐ Confirmar que ele ainda cria e edita as próprias broncas mesmo sem permissão de Moderação.
- ☐ Desfazer as regras de teste ao final.

---

## Se algo der errado

Uma migração falhou no meio. O db push para na migração com erro; as anteriores já foram aplicadas. Corrija a causa (normalmente dados inconsistentes, veja 2.4) e rode db push de novo - ele retoma de onde parou.

Gravações começaram a falhar após a 165. Verifique se a função existe:

```
select public.can_write(auth.uid(), 'works');
```

Se a função não existir, a 164 não foi aplicada - aplique-a antes da 165.

Em emergência, esvaziar permission\_rules devolve o acesso a todos, já que o padrão sem regras é liberado:

```
delete from public.permission_rules;
```

Reverter o código. O deploy do Vercel pode voltar para a publicação anterior pelo painel (Deployments -> ... -> Promote to Production). As migrações não são revertidas junto - mas o app antigo continua funcionando com o banco novo, porque todas as mudanças são aditivas.

---

## Resumo da ordem

1. Backup de produção
2. `npx supabase db push` (migrações 141->165)
3. `yarn deploy:functions:nacionalizacao` + conferir secrets do Resend
4. Merge da branch em main (Vercel publica sozinho)
5. Rodar a checklist de verificação
6. Opcional: gerar APK/AAB com `yarn build:prod`

---

## Referência rápida dos scripts

| Comando                                           | Quando usar                                            |
|---------------------------------------------------|--------------------------------------------------------|
| <code>npx supabase db push</code>                 | Aplicar as migrações (Passo 2)                         |
| <code>yarn deploy:functions:nacionalizacao</code> | Publicar as 4 functions desta leva                     |
| <code>yarn build:prod</code>                      | APK release apontando para produção                    |
| <code>yarn build:prod:aab</code>                  | AAB para a Play Store (incrementa versionCode)         |
| <code>yarn build:standalone:dev</code>            | APK de teste apontando para o banco de dev             |
| <code>yarn build</code>                           | Só o bundle web (produção) - o Vercel faz isso sozinho |
| <code>yarn build:dev</code>                       | Só o bundle web apontando para dev                     |

Não precisa rodar:

- `yarn build` / `yarn build:clean` - o Vercel roda no servidor ao detectar o merge. Só use localmente para conferir que o build passa.
- `deploy:functions:prod` - publica outro conjunto de funções (push, pagamentos, compartilhamento), nenhuma delas alterada nesta leva.
- `import:poles` - importação pontual de postes por KMZ, não faz parte deste deploy.
- Qualquer `build:standalone*` - só se for publicar uma nova versão do app Android. O site não depende disso.